QALight

Frontend-розробка

Заняття 2. Система контролю версій Git

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Система контролю версій

Створення репозиторію на GitHub

Персональний сайт на github

Система контролю версій

Теорія: Version Control System

Система контролю версій (VCS - Version Control System) - це система, що записує зміни у набір файлів протягом деякого часу, що дозволяє повернутися до певної версії пізніше.

Багато людей в якості одного з методів контролю версій застосовують копіювання файлів в окрему директорію. Даний підхід є дуже поширеним завдяки його простоті, проте він схильний до появи помилок. Можна легко забути в якій директорії ви знаходитеся і випадково змінити не той файл або скопіювати не ті файли, які ви хотіли. У 2005 році спільнотою розробників ядра Linux була розроблена власна **децентралізована система контролю версій - Git**.

За допомогою **системи контролю версій** Git клієнти не просто отримують останній знімок файлів репозиторія, а й повну копію сховища разом з усією його історією. Система дуже добре взаємодіє з кількома віддаленими репозиторіями, що дає можливість співпрацювати з різними групами людей, застосовуючи різні підходи в межах одного проекту одночасно. Це дозволяє налаштувати декілька типів робочих процесів, таких як ієрархічні моделі.

Інсталяція Git

Початкове налаштування Git

Отримання допомоги

Ініціалізація репозиторія

Відстеження файлів

Інсталяція Git

Практика: Інсталяція Git

Інсталяція на Linux через основний менеджер управління пакетами.

Якщо ви використовуєте будь-який споріднений дистрибутив на базі RPM, скористайтеся dnf:

$ **sudo dnf install git-all**

Якщо ви використовуєте Debian-подібний дистрибутив, такий як Ubuntu, спробуйте apt-get:

$ **sudo apt-get install git-all**

Інсталяція на Mac за допомогою бінарного інсталятору для macOS.

Інсталяція на Windows - перейдіть до http://git-scm.com/download/win і завантаження почнеться автоматично.

Початкове налаштування Git

Практика: Початкове налаштування Git

Git має утиліту **git config**, яка дозволяє отримати чи встановити параметри, що контролюють усіма аспектами того, як Git виглядає чи працює.

Перше, що ви повинні зробити - це встановити ім’я користувача та адресу електронної пошти. Кожен commit в Git використовує цю інформацію, і вона незмінно включена у commit:

git config --global user.name "John Doe**"**

git config --global user.email johndoe@example.com

Якщо ви передаєте опцію **--global**, ці налаштування потрібно зробити **тільки один раз**. Якщо ви хочете, перевизначити ім’я або адресу електронної пошти для конкретних проектів, ви можете виконати цю ж команду без опції --global в каталозі необхідного проекту.

Перевірка налаштувань

Практика: Початкове налаштування Git

Можете скористатися командою git config --list, щоб переглянути всі налаштування Git:

```
git config --list
user.name=John Doe
user.email=johndoe@example.com
color.status=auto
color.branch=auto
color.interactive=auto
color.diff=auto

...
```

Ви також можете перевірити значення конкретного ключа, виконавши git config <key>:

```
$ git config user.name
John Doe
```

Отримання допомоги

Практика: Початкове налаштування Git

Якщо вам знадобиться допомога при використанні Git, є два еквівалентних способи, щоб отримати допомогу на докладних сторінках довідника (manpage) для будь-якої команди Git:

$ git help <verb>

$ man git-<verb>

Наприклад, ви можете отримати допомогу у довіднику для команди git config, виконавши:

$ **git help config**

Крім того, якщо вам не потрібен вичерпний довідник, а треба лише трохи освіжити пам’ять щодо доступних опцій команди git, ви можете отримати стислішу версію за допомогою опцій -h чи --help, наприклад:

$ **git add -h**

Ініціалізація репозиторія

Практика: Ініціалізація репозиторія

Зазвичай Git репозиторій отримують одним з двох способів:

- Беруть локальну директорію, що наразі не під контролем версій, та перетворюють її на сховище Git, або
- Звідкілясь клонують існуючий Git репозиторій.

Створіть десь ваш проект: mkdir <your project>

Якщо у вас вже є тека з проектом, що наразі не перебуває під контролем версії, і ви бажаєте почати використовувати з цим проектом Git, спочатку треба перейти до теки цього проекту:
для Linux: $ cd /home/user/<your project>
для Mac: $ cd /Users/user/<your project>
для Windows: $ cd /c/user/<your project>

та виконати: git init

Це створить новий підкаталог .git, який містить всі необхідні файли вашого репозиторія - скелет Git-репозиторію.

Відстеження файлів

Практика: Відстеження файлів

Якщо ви бажаєте додати існуючі файли під версійний контроль (на відміну від порожнього каталогу), ймовірно, вам слід проіндексувати ці файли і зробити перший коміт. Ви можете це зробити за допомогою декількох команд **git add**, що визначають файли, за якими ви бажаєте слідкувати, після яких треба виконати **git commit**:

$ git add .

$ git add README.md

$ git commit -m 'Перша версія проекту'

Наразі, у вас є Git репозиторій з декількома файлами та першим комітом.

Реєстрація на GitHub

Облікові записи
Облікові записи користувачів
Плани GitHub
Особистий обліковий запис
Індивідуальні плани та переваги GitHub Copilot

Реєстрація на GitHub

Теорія: Реєстрація на GitHub

GitHub - веб-сервіс, що ґрунтується на системі Git.

Це соціальна мережа для розробників, яка допомагає зручно вести колективну розробку IT-проектів. Тут можна публікувати та редагувати свій код, коментувати чужі наробки, спостерігати за новинами від інших користувачів, тощо.

Зареєструйтесь на https://github.com/ та створіть свій репозиторій **<your repo>**.

Додайте **remote** репозиторій до вашого проекту:

```
git remote add origin https://github.com/<your name>/<your repo>.git
```

Облікові записи

Теорія: Реєстрація на GitHub

За допомогою GitHub ви можете зберігати код та співпрацювати над ним. Облікові записи дозволяють вам організовувати та контролювати доступ до цього коду.

На GitHub є **три типи** облікових записів.

1. Облікові записи користувачів
2. Облікові записи організацій
3. Облікові записи підприємств

Кожна людина, яка використовує GitHub, входить в обліковий запис користувача.

Обліковий запис організації покращує співпрацю між кількома користувачами, а обліковий запис підприємства дозволяє централізовано керувати кількома організаціями.

Облікові записи користувачів

Теорія: Реєстрація на GitHub

Кожна людина, яка використовує GitHub, входить в обліковий запис користувача.
Ваш обліковий запис користувача – це ваша ідентифікація на GitHub, і він має ім'я користувача та профіль.
Обліковий запис користувача може володіти такими ресурсами, як репозиторії, пакети та проекти.
Щоразу, коли ви виконуєте будь-яку дію на GitHub, наприклад, створюєте проблему або переглядаєте запит на зняття змін, ця дія приписується вашому обліковому запису користувача.
Облікові записи користувачів призначені для людей, але ви можете створювати облікові записи для автоматизації діяльності на GitHub. Цей тип облікового запису називається **користувачем машини**.
Наприклад, ви можете створити обліковий запис користувача машини для автоматизації робочих процесів **безперервної інтеграції** (**CI**). Існує два типи облікових записів користувачів:

- Особисті облікові записи
- Керовані облікові записи користувачів

Плани GitHub

Теорія: Реєстрація на GitHub

GitHub пропонує безкоштовні та платні плани для зберігання та спільної роботи над кодом. Деякі плани доступні лише для особистих облікових записів, тоді як інші плани доступні лише для організаційних та корпоративних облікових записів.
Завдяки **GitHub Free** для особистих облікових записів ви можете працювати з необмеженою кількістю співавторів у необмеженій кількості публічних репозиторіїв з повним набором функцій та в необмеженій кількості приватних репозиторіїв з обмеженим набором функцій. GitHub Free особистий обліковий запис включає:

- Підтримка спільноти GitHub
- Сповіщення Dependabot
- Правила захисту розгортання для публічних репозиторіїв
- Застосування двофакторної автентифікації
- 500 МБ сховища пакетів GitHub
- 120 годин використання кодових просторів GitHub на місяць
- 15 ГБ сховища кодових просторів GitHub на місяць
- Функції GitHub Actions: 2000 хвилин на місяць Правила захисту розгортання для публічних репозиторіїв Сторінки GitHub у публічних репозиторіях

Особистий обліковий запис

Теорія: Реєстрація на GitHub

Щоб розпочати роботу з GitHub, вам потрібно створити безкоштовний особистий обліковий запис і підтвердити свою адресу електронної пошти.
Ви також можете пройти автентифікацію за допомогою Google або Apple – підтримуваних постачальників соціальних мереж для входу, коли створюєте свій обліковий запис на GitHub. Для користувачів iOS, навіть якщо ви ввімкнули налаштування «Приховати мої адреси електронної пошти» для свого облікового запису Apple, використання соціального входу призведе до створення нового облікового запису GitHub.

Кожна людина, яка використовує GitHub, входить до облікового запису користувача. Ваш обліковий запис користувача – це ваша особистість на GitHub і має ім’я користувача та профіль.

Керовані облікові записи користувачів

Теорія: Реєстрація на GitHub

Якщо ваш обліковий запис було створено для вашого підприємства на GitHub Enterprise Cloud, ви використовуєте **керований обліковий запис користувача.**

Деякими даними та налаштуваннями вашої професійної записи керує ваше підприємство. Ви повинні увійти до свого керованого професійного запису користувача, щоб отримати доступ до організацій та репозиторіїв, що належать підприємству.
Ви можете створювати власні приватні репозиторії, але не можете створювати публічний контент або робити внесок у репозиторії поза підприємством.

Реєстрація нового особистого облікового запису

Теорія: Реєстрація на GitHub

Перейдіть за посиланням https://github.com/. Натисніть «Зареєструватися». Або натисніть «Продовжити з Google», щоб зареєструватися за допомогою соціального входу.
Дотримуйтесь інструкцій, щоб створити свій особистий обліковий запис.
Під час реєстрації вам буде запропоновано підтвердити свою адресу електронної пошти. Без підтвердженої адреси електронної пошти ви не зможете виконувати деякі основні завдання GitHub, такі як створення репозиторію.
Деякі підприємства створюють керовані облікові записи користувачів для своїх користувачів. Ви не можете зареєструвати особистий обліковий запис з адресою електронної пошти, яка вже підтверджена для керованого облікового запису користувача. Якщо у вас виникли проблеми з підтвердженням адреси електронної пошти, є кілька кроків для усунення несправностей, які ви можете виконати.
Ми наполегливо рекомендуємо налаштувати 2FA для вашого облікового запису. 2FA — це додатковий рівень безпеки, який може допомогти захистити ваш обліковий запис.

Індивідуальні плани та переваги GitHub Copilot

Теорія: Реєстрація на GitHub

GitHub пропонує кілька планів **Copilot** для індивідуальних розробників, кожен з яких має різні функції, доступ до моделей та обмеження використання для підтримки широкого спектру потреб кодування.

GitHub Copilot Free - Для розробників, які хочуть розпочати роботу з Copilot.

- Включає до 2000 завершень коду та певну кількість кредитів GitHub AI
- Обмежене використання чату та агента з моделями, доступними лише через автоматичний вибір моделі

Розроблено, щоб дати вам обмежений досвід у можливостях Copilot
Не потрібна підписка чи оплата Призначено лише для особистого використання, не для користувачів, якими керує організація чи підприємство

Чудово підходить для розробників, які хочуть дослідити можливості Copilot перед переходом на платний план

Створення репозиторію

Створення репозиторію. Створення гілки
Внесення та фіксація змін
Запит на злиття
Цикл життя статусу файлів
Ігнорування файлів
Збереження змін

Створення репозиторію

Практика: Створення репозиторію

Репозиторій зазвичай об’єднує елементи, що належать до одного «проекту» або предмета, над яким ви працюєте.
Часто репозиторії містять файл **README**, файл з інформацією про ваш проект. Файли README написані мовою **Markdown**, яка є легкою для читання та написання мовою для форматування звичайного тексту. GitHub дозволяє додавати файл README одночасно зі створенням нового репозиторію. GitHub також пропонує інші поширені опції, такі як файл ліцензії.
Ваш репозиторій може бути місцем, де ви зберігаєте ідеї, ресурси або навіть обмінюєтеся інформацією та обговорюєте її з іншими.

Створення репозиторію :

- У верхньому правому куті будь-якої сторінки виберіть , а потім натисніть «Новий репозиторій».
- У полі «Назва репозиторію» введіть **<repo name>** .
- У полі «Опис» введіть короткий опис. Наприклад, введіть «Цей репозиторій призначений для практики роботи з GitHub».
- Виберіть, чи буде ваш репозиторій публічним, чи приватним.
- Виберіть «Додати файл README». Натисніть «Створити репозиторій».

Створення гілки

Практика: Створення гілки

Розгалуження дозволяє вам мати різні версії репозиторію одночасно. За замовчуванням ваш репозиторій має одну гілку під назвою **main**, яка вважається остаточною гілкою.
Ви можете створювати додаткові гілки з main у своєму репозиторії. Розгалуження корисне, коли ви хочете додати нові функції до проекту, не змінюючи основного вихідного коду. Робота, виконана над різними гілками, не відображатиметься в основній гілці, доки ви не об'єднаєте її.
Ви можете використовувати гілки для експериментів та внесення змін перед тим, як закомітити їх до основної гілки. Коли ви створюєте гілку з основної гілки, ви створюєте копію або знімок основної гілки, якою вона була на той момент часу. Якщо хтось інший вніс зміни до основної гілки, поки ви працювали над нею, ви можете отримати ці оновлення.

Створення гілки
Натисніть вкладку **«Код»** вашого репозиторію. Над списком файлів натисніть спадне меню з написом main.
Введіть назву гілки у текстове поле. Натисніть «Створити гілку»..

Внесення та фіксація змін

Практика: Внесення та фіксація змін

Коли ви створили нову гілку на попередньому кроці, GitHub перенаправив вас на сторінку коду вашої нової гілки, яка є копією main. Ви можете вносити та зберігати зміни до файлів у вашому репозиторії. На GitHub збережені зміни називаються **комітами**. Кожен коміт має пов'язане повідомлення коміту, яке є описом, що пояснює, чому була внесена певна зміна. Повідомлення коміту фіксують історію ваших змін, щоб інші учасники могли зрозуміти, що ви зробили і чому.
У створеній вами гілці клацніть файл README.md.
У редакторі напишіть трохи про себе. Натисніть «Зафіксувати зміни». У полі «Зафіксувати зміни» напишіть повідомлення коміту, яке описує ваші зміни. Натисніть «Зафіксувати зміни». Ці зміни будуть внесені лише до файлу README у вашій гілці, тому тепер ця гілка містить контент, який відрізняється від main.

Pull requests

Практика: Внесення та фіксація змін

Тепер, коли у вас є зміни в гілці поза основною, ви можете відкрити **запит на зняття**. Запити на зняття показують відмінності або відмінності контенту з обох гілок. Зміни, доповнення та вилучення відображаються різними кольорами. Щойно ви зробите коміт, ви можете відкрити запит на зняття та розпочати обговорення, ще до того, як код буде завершено. На цьому кроці ви відкриєте запит на зняття у власному репозиторії, а потім самостійно його об'єднаєте.

- Перейдіть на вкладку Запити на зняття у вашому репозиторії.
- Натисніть Новий запит на зняття. У полі Приклади порівнянь виберіть створену вами гілку для порівняння з основною (оригіналом).
- Перегляньте свої зміни в різницях на сторінці порівняння та переконайтеся, що вони відповідають тому, що ви хочете надіслати.
- Дайте своєму запиту на втягування назву та напишіть короткий опис ваших змін.
- Натисніть «Створити запит на втягування».

Коли ви починаєте співпрацювати з іншими, саме тоді ви просите їх про перевірку. Це дозволяє вашим співробітникам коментувати або пропонувати зміни до вашого запиту на втягування, перш ніж ви об'єднаєте зміни в основну гілку.

Запит на злиття

Практика: Внесення та фіксація змін

Об'єднайте ваш запит на злиття На цьому останньому кроці ви об'єднаєте свою гілку readme-edits з гілкою main. Після об'єднання запиту на злиття зміни у вашій гілці будуть включені до гілки main. Іноді запит на злиття може вносити зміни до коду, які конфліктують з існуючим кодом на гілці main. Якщо є якісь конфлікти, GitHub попередить вас про конфліктний код і запобігатиме об'єднанню, доки конфлікти не будуть вирішені. Ви можете зробити коміт, який вирішить конфлікти, або використовувати коментарі в запиті на злиття, щоб обговорити конфлікти з членами вашої команди. Унизу запиту на злиття натисніть «Об'єднати запит на злиття», щоб об'єднати зміни з гілкою main. Натисніть «Підтвердити злиття». Ви отримаєте повідомлення про те, що запит успішно об'єднано, а запит закрито. Натисніть «Видалити гілку».
Тепер, коли ваш запит на злиття об'єднано, а ваші зміни знаходяться на гілці main, ви можете безпечно видалити гілку. Якщо ви хочете внести більше змін до свого проєкту, ви завжди можете створити нову гілку та повторити цей процес. Поверніться на вкладку «Код» вашого репозиторію, щоб побачити опубліковані зміни в основній гілці.

Клонування репозиторія

Практика: Внесення та фіксація змін

Якщо ви бажаєте отримати копію існуючого Git репозиторію - вам потрібна команда **git clone**. Git отримує повну копію майже всіх даних, що є у сервера. Кожна версія кожного файлу в історії проекту витягується автоматично, коли ви виконуєте git clone. Насправді, якщо щось станеться з диском вашого серверу, ви зазвичай можете використати майже будь-який з клонів на будь-якому клієнті щоб повернути сервер до стану на момент клонування.

Щоб клонувати репозиторій треба використати команду git clone <url>:

$ git clone https://github.com/ <name>/<repo>

Це створить директорію під назвою **<repo>** , проведе ініціалізацію директорії .git, забере всі дані для репозиторія, та приведе директорію до стану останньої версії.

Якщо ви бажаєте зробити клон репозиторія в директорію з іншою назвою, ви можете передати її як другий параметр команди:

$ git clone https://github.com/ <name>/<repo> **myrepo**

Запис змін до репозиторію

Практика: Внесення та фіксація змін

Зазвичай, ви хочете зробити деякі зміни та записати їх у вашому репозиторії кожного разу, коли ваш проект набуває стану, що ви бажаєте зберегти.

Кожен файл вашої робочої директорії може бути в одному з двох станів: контрольований (**tracked**) чи неконтрольований (**untracked**). Контрольовані файли - це файли, що були в останньому знімку. Вони можуть бути не зміненими, зміненими або індексованими. Якщо стисло, контрольовані файли - це файли, про які Git щось знає.

Неконтрольовані файли - це все інше, будь-які файли у вашій робочій директорії, що не були у вашому останньому знімку та не існують у вашому індексі. Якщо ви щойно зробили клон репозиторія, усі ваші файли контрольовані та не змінені, адже Git щойно їх отримав, а ви нічого не редагували.

По мірі редагування файлів, Git бачить, що вони змінені, адже ви їх змінили після останнього коміту. Впродовж роботи ви вибірково індексуєте змінені файли і потім зберігаєте всі індексовані зміни, далі цикл повторюється.

Цикл життя статусу файлів

Практика: Цикл життя статусу файлів

Щоб дізнатись, в якому стані ваші файли, варто скористатись командою **git status**. Якщо ви виконуєте цю команду відразу після клонування, ви маєте побачити таке:

```
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
nothing to commit, working directory clean
```

Це означає, що жоден з контрольованих файлів не змінено. Припустімо, ви додали новий файл README до вашого проекту. Якщо файл раніше не існував, і ви виконаєте git status, ви побачите ваш неконтрольований файл так:

```
On branch master
Your branch is up-to-date with 'origin/master'.
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README
nothing added to commit but untracked files present (use "git add" to track)
```

Новий файл README неконтрольований, адже він під заголовком “Untracked files” у статусі.

Контролювання нових файлів

Практика: Version Control System

Щоб почати контролювати новий файл, вам треба використати команду **git add**:

git add README

Якщо ви знову виконаєте команду status, ви побачите, що ваш файл README тепер контролюється та готовий до включення до коміту:

```
On branch master
Your branch is up-to-date with 'origin/master'.
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    new file:   README
```

Файл доданий, бо він під заголовком “Changes to be committed”. Якщо ви створите коміт зміни зараз, версія файлу на момент коли ви виконали git add буде збережена в знімку в історії. Команда git add приймає шлях до файлу або директорії. Якщо це директорія, команда додає усі файли в цій директорії рекурсивно.

Ігнорування файлів

Практика: Ігнорування файлів

Якщо є файли, що не підлягають автоматичному індексуванню, ви можете створити файл **.gitignore**, що містить **взірці**, яким відповідають ці файли. GitHub підтримує доволі вичерпний список прикладів файлів .gitignore для різноманітних проектів та мов програмування. За адресою https://github.com/github/gitignore можна отримати зразок файлу **gitignore** для свого проекту.

Стандартні ґлоб взірці працюють, і будуть застосовані для всього робочого дерева рекурсивно..

Ґлоб (glob) взірці – це ніби спрощені регулярні вирази, що їх використовують оболонки. Зірочка (*) відповідає нулю або більше символам. [абв] відповідає будь-якому з символів всередині квадратних дужок (у цьому випадку а, б або в). Знак питання (?) відповідає одному символу. Квадратні дужки з символами, що розділені дефісом ([0-9]) відповідають будь-якому символу між ними (у даному випадку від 0 до 9). Ви можете використовувати дві зірочки щоб позначити вкладені директорії: a/**/z відповідає a/z, a/b/z, a/b/c/z тощо.

Збереження змін

Практика: Збереження змін

Найпростіший спосіб зберегти зміни - виконати git commit: **git commit.** Можна створити повідомлення при виконанні команди commit за допомогою опції -m:

git commit **-m** "Story 182: Fix benchmarks for speed"

Щоразу ви створюєте commit, ви записуєте знімок вашого проекту, до якого ви можете повернутися пізніше.

Ви можете виконати наступне для надсилання всіх зроблених змін до сервера:

git push origin master

Щоб дізнатись інформацію щодо окремого віддаленого сховища, використовують команду git remote show <назва сховища>.:

git remote show origin

Вона виводить посилання для віддаленого сховища, а також інформацію про відстеження за гілками.

Персональний сайт на github

Персональний сайт на github

Практика: Персональний сайт на github

GitHub Pages доступний у публічних репозиторіях з GitHub Free та GitHub Free для організацій, а також у публічних та приватних репозиторіях з GitHub Pro, GitHub Team, GitHub Enterprise Cloud та GitHub Enterprise Server. Дивіться плани GitHub.
Ви можете створити репозиторій або вибрати існуючий репозиторій для свого сайту.

Якщо ви хочете створити сайт GitHub Pages для репозиторію, де не всі файли в репозиторії пов'язані з сайтом, ви зможете налаштувати джерело публікації для свого сайту. Наприклад, ви можете мати спеціальну гілку та папку для зберігання вихідних файлів вашого сайту або ви можете використовувати власний робочий процес GitHub Actions для створення та розгортання вихідних файлів вашого сайту.

Якщо обліковий запис, якому належить репозиторій, використовує GitHub Free або GitHub Free для організацій, репозиторій має бути публічним.

Персональний сайт у існуючому репозиторії

Практика: Персональний сайт на github

Якщо ви хочете створити сайт у існуючому репозиторії, перейдіть до розділу «Створення сайту».

У верхньому правому куті будь-якої сторінки виберіть , а потім натисніть «Новий репозиторій».
Використовуйте випадаюче меню «Власник», щоб вибрати обліковий запис, якому ви хочете надати репозиторій.
Введіть назву для вашого репозиторію та необов’язковий опис. Якщо ви створюєте сайт користувача або організації, ваш репозиторій повинен мати назву <користувач>.github.io або <організація>.github.io. Якщо ім’я вашого користувача або організації містить великі літери, ви повинні писати їх малими.
Виберіть видимість репозиторію.
Увімкніть опцію "Додати README".
Натисніть "Створити репозиторій"

Створення сайту

Практика: Персональний сайт на github

Перш ніж створювати сайт, у вас має бути репозиторій для вашого сайту на GitHub. Якщо ви не створюєте свій сайт у існуючому репозиторії, див. розділ Створення репозиторію для вашого сайту.

Попередження

Сайти сторінок GitHub є загальнодоступними в Інтернеті, навіть якщо репозиторій для сайту є приватним (якщо ваш план або організація це дозволяє). Якщо у репозиторії вашого сайту є конфіденційні дані, ви можете видалити їх перед публікацією. Для отримання додаткової інформації див. розділ Про репозиторії.

На GitHub перейдіть до репозиторію вашого сайту.

Визначте, яке джерело публікації ви хочете використовувати. Див. розділ Налаштування джерела публікації для вашого сайту сторінок GitHub.

Перегляд опублікованого сайту

Практика: Персональний сайт на github

Під назвою вашого репозиторію натисніть «Налаштування». Якщо вкладка «Налаштування» не відображається, виберіть випадаюче меню та натисніть «Налаштування».

Знімок екрана заголовка репозиторію, на якому показано вкладки. Вкладка «Налаштування» виділена темно-помаранчевим контуром.

У розділі «Код та автоматизація» на бічній панелі натисніть «Сторінки».

Щоб переглянути опублікований сайт, у розділі «Сторінки GitHub» натисніть «Відвідати сайт».

Примітка

Після внесення змін до GitHub публікація змін на вашому сайті може тривати до 10 хвилин. Якщо ви не бачите змін на сайті GitHub Pages у браузері протягом години, див. розділ «Про помилки збірки Jekyll для сайтів GitHub Pages».

Персональний сайт у існуючому репозиторії

Практика: Персональний сайт на github

Створіть новий репозиторій з ім’ям свого облікового запису.

Наприклад, якщо мій обліковий запис couchjanus - Repository name couchjanus.github.io

В локальному директорії проекту виконайте команду:

```
git remote add origin https://github.com/couchjanus/ couchjanus.github.io.git
```

Створіть файл **README.md**

Створіть файл **index.html**

Виконайте команди:

```
git add .
git commit -m "first commit"
git push origin main
```

В браузере викличте https://couchjanus.github.io

ВПРАВИ

Вправи до цієї лекції знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers
https://developer.mozilla.org/en-US/docs/Web

Система контролю версій https://git-scm.com/

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua